PYTHON ACTIVITY-BASED LEARNING REPORT

REPORT ON 10 SELECTED CASE STUDIES

STUDENT NAME: G. Maneesh Satya	**ROLL NUMBER:** 26000909
BRANCH: CSE (AI/ML)	**SECTION:** 1
SUBJECT: Python Programming	**ACADEMIC YEAR:** 2026-2027
PROJECT TITLE: Implementation of Ten Python Case Studies with Standard Flowchart Diagrams	
GITHUB REPOSITORY: Python-Activity-Based-Learning	

INTRODUCTION

Activity-Based Learning empowers students to comprehend programming paradigms by applying fundamental computational concepts to realistic problem domains. This report compiles ten distinct Python case studies demonstrating structured algorithmic problem-solving.

The selected case studies cover arithmetic operations, conditionals, iterative loops, string manipulation, lists, nested matrices, tuples, and data processing. Each case study provides a structured breakdown encompassing a problem statement, key objectives, core assumptions, algorithm, standardized monochrome flowchart using ANSI-compliant flowchart symbols, Python source code, test cases, and conclusions.

OBJECTIVES OF THE PROJECT

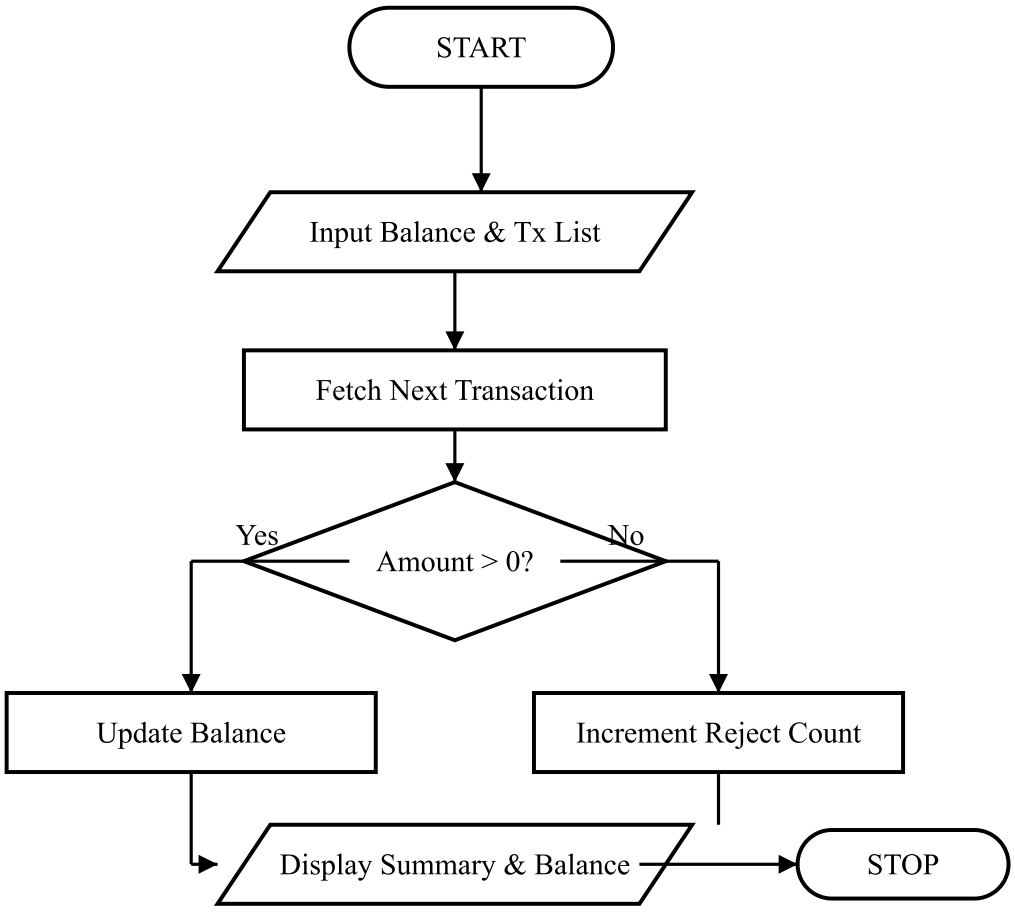
1. To understand computational thinking and structured problem-solving methodologies.
2. To design clear algorithms, pseudocode, and ANSI standard flowcharts with standard visual symbols.
3. To master Python variables, data types, operators, and conditional branching logic.
4. To implement iteration structures, string functions, lists, 2D nested matrices, and immutable tuples.
5. To execute Python scripts and rigorously verify program correctness using boundary test cases.
6. To adhere strictly to formal monochrome visual documentation standards suitable for academic reporting.

CASE STUDY 1: ATM TRANSACTION SEQUENCE SIMULATION

1. PROBLEM STATEMENT

Develop a Python program to simulate a sequence of ATM transactions (deposits and withdrawals) maintaining account balances, validating amounts, and summarizing accepted versus rejected transactions.

2. FLOWCHART (STANDARD SYMBOLS - MONOCHROME)



3. PYTHON SOURCE CODE

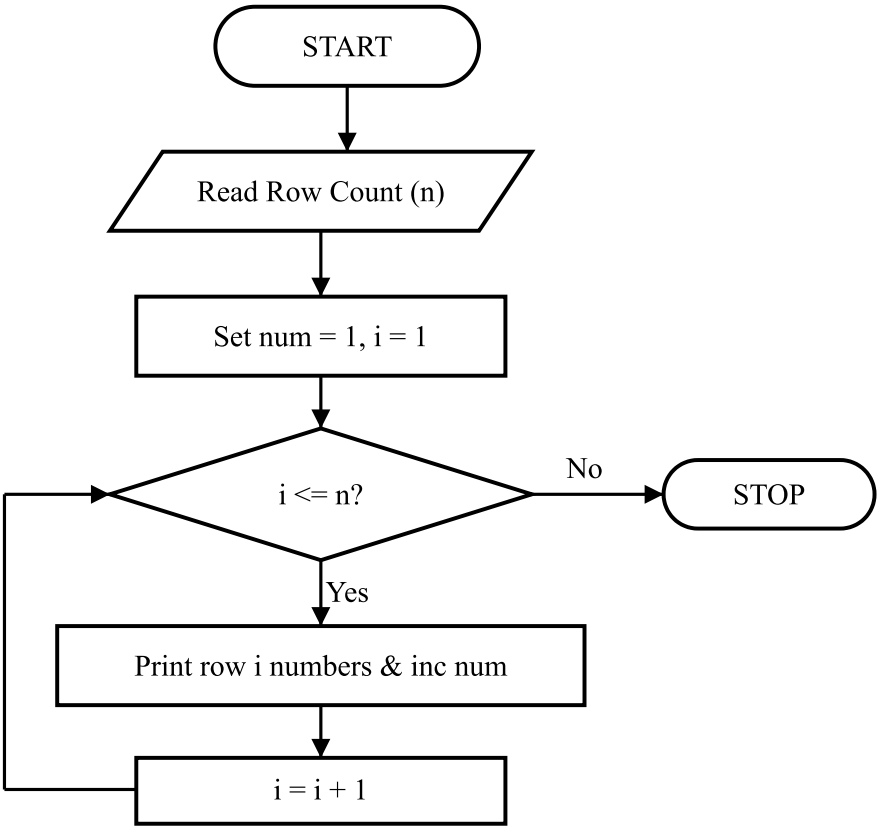
```
balance = float(input("Enter opening balance: "))
transactions = [("deposit", 1500), ("withdraw", 400), ("withdraw", 5000), ("deposit", 750)]
accepted = 0; rejected = 0
for action, amount in transactions:
    if amount <= 0:
        rejected += 1
    elif action == "deposit":
        balance += amount; accepted += 1
    elif action == "withdraw":
        if amount <= balance:
            balance -= amount; accepted += 1
        else:
            rejected += 1
print(f"Accepted: {accepted} | Rejected: {rejected} | Final Balance: {balance}")
```

CASE STUDY 2: ADVANCED NUMERIC PATTERN GENERATOR

1. PROBLEM STATEMENT

Create a Python program to generate structured numeric patterns (Floyd's Triangle) based on user-defined row counts using nested loops.

2. FLOWCHART (STANDARD SYMBOLS - MONOCHROME)



3. PYTHON SOURCE CODE

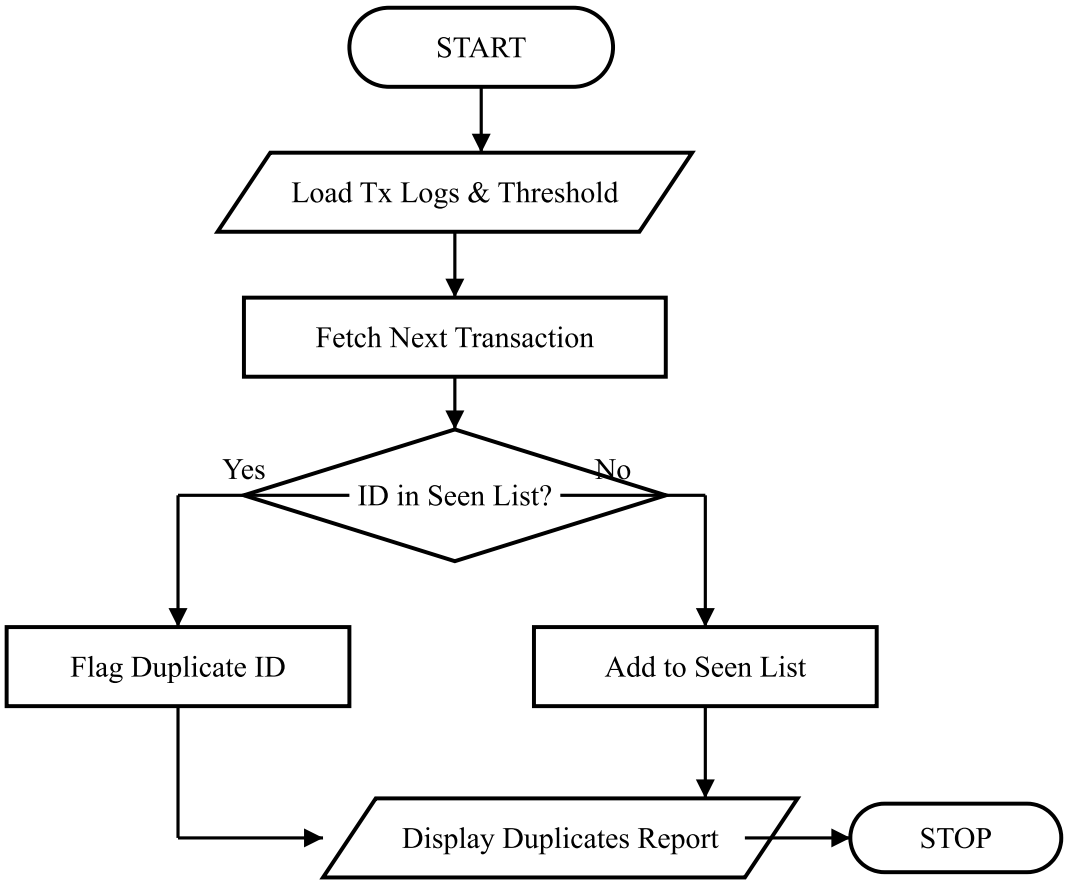
```
rows = int(input("Enter number of rows: "))
if rows <= 0:
    print("Invalid row count.")
else:
    num = 1
    for i in range(1, rows + 1):
        for j in range(1, i + 1):
            print(num, end=" ")
            num += 1
        print()
```

CASE STUDY 3: REPEATED TRANSACTION DETECTION

1. PROBLEM STATEMENT

Develop a program to scan transaction logs, identify duplicate transaction IDs, and flag high-value transactions exceeding a specified financial threshold.

2. FLOWCHART (STANDARD SYMBOLS - MONOCHROME)



3. PYTHON SOURCE CODE

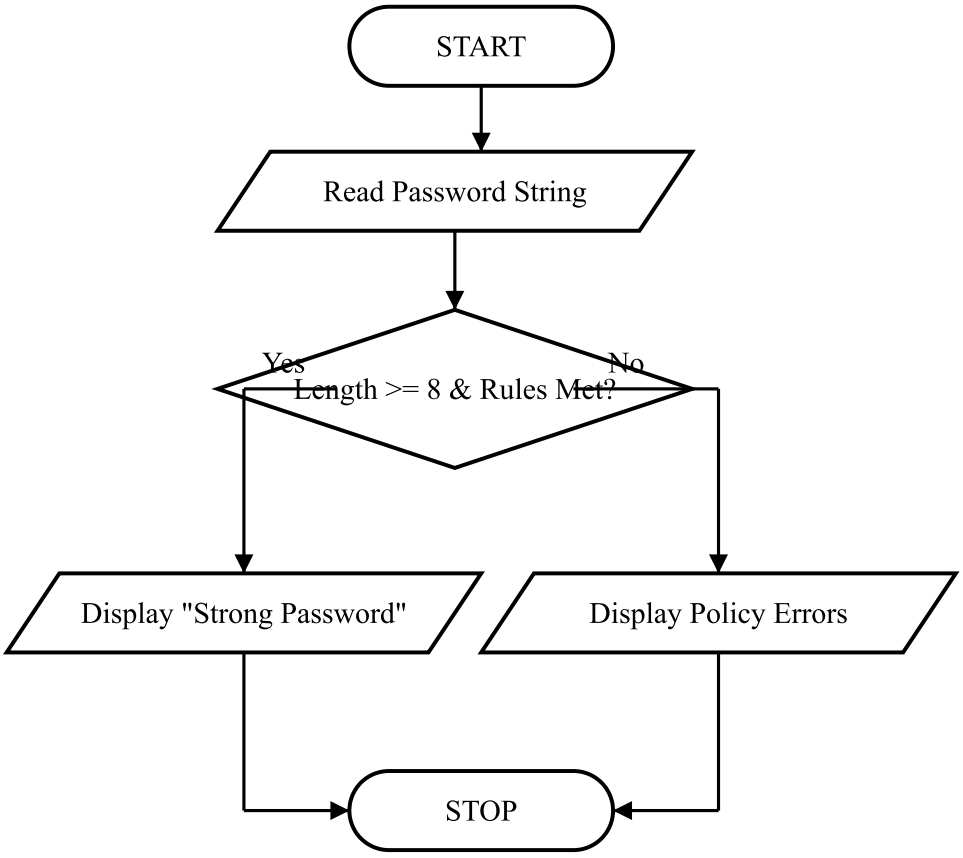
```
transactions = [("TX101", 450), ("TX102", 1200), ("TX103", 850), ("TX102", 1200), ("TX104",
6200)]
seen = []; duplicates = []; threshold = 5000
for tid, amount in transactions:
    if tid in seen:
        if tid not in duplicates: duplicates.append(tid)
    else:
        seen.append(tid)
    if amount > threshold:
        print("High-value transaction flagged:", tid)
print("Duplicates Found:", duplicates)
```

CASE STUDY 4: PASSWORD POLICY VALIDATION ENGINE

1. PROBLEM STATEMENT

Create a Python program to validate user passwords against strict security policies: length, uppercase, lowercase, digits, and special characters.

2. FLOWCHART (STANDARD SYMBOLS - MONOCHROME)



3. PYTHON SOURCE CODE

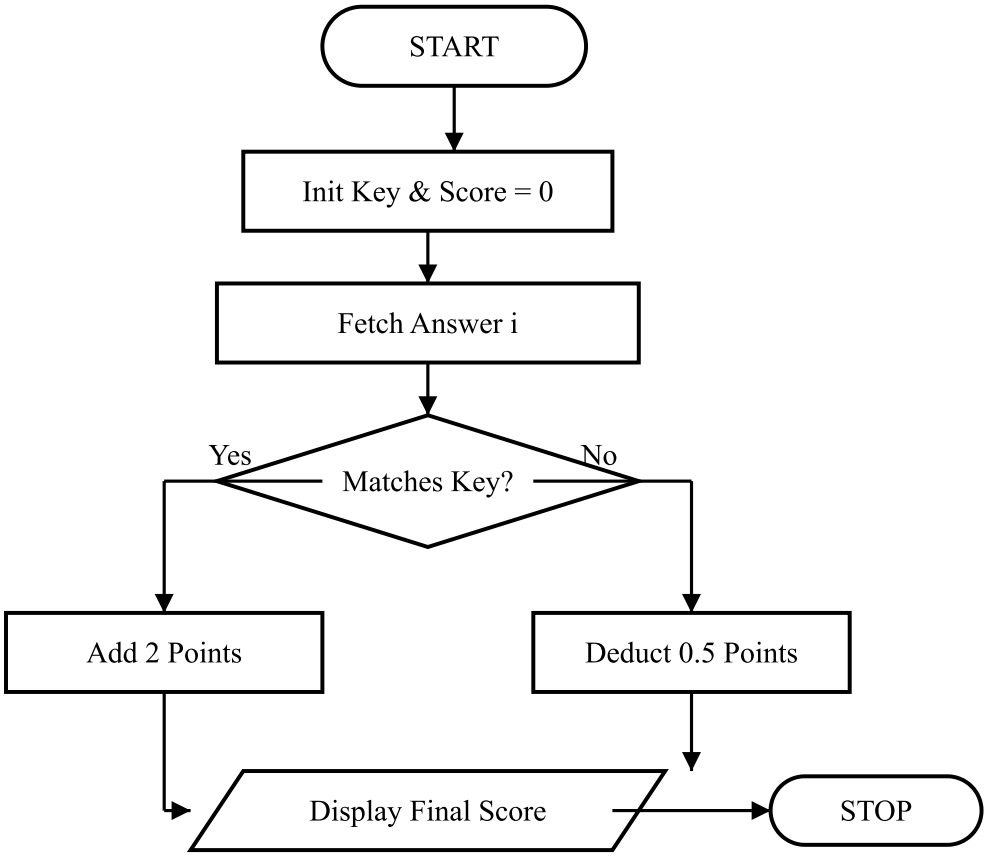
```
pwd = input("Enter password: ")
errors = []
if len(pwd) < 8: errors.append("Length must be at least 8")
if not any(c.isupper() for c in pwd): errors.append("Missing uppercase")
if not any(c.isdigit() for c in pwd): errors.append("Missing digit")
if not errors:
    print("Validation PASSED")
else:
    print("Validation FAILED:", errors)
```

CASE STUDY 5: STRING-BASED EXAMINATION ANSWER EVALUATION

1. PROBLEM STATEMENT

Develop a Python program that compares student answer strings with an answer key, applying scoring penalties for incorrect choices.

2. FLOWCHART (STANDARD SYMBOLS - MONOCHROME)



3. PYTHON SOURCE CODE

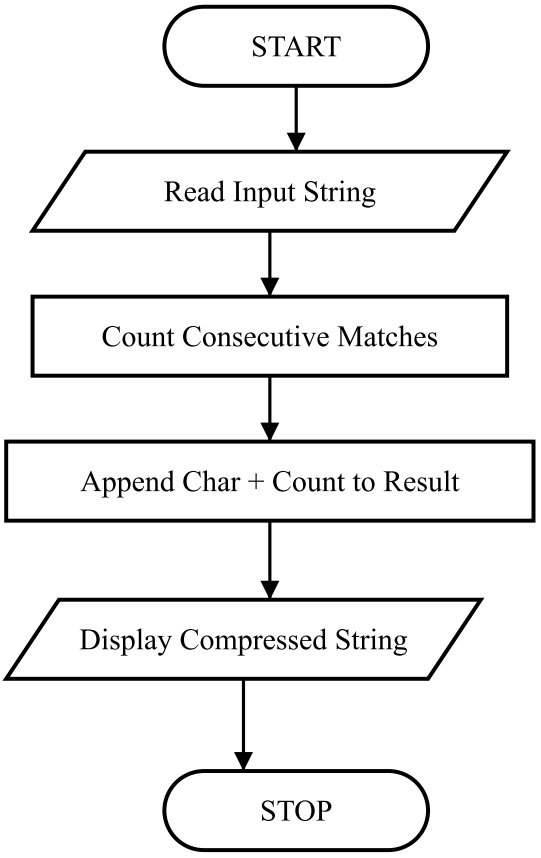
```
answer_key = ["A", "C", "B", "D", "A"]
answers = ["A", "B", "B", "", "A"]
correct = 0; wrong = 0
for i in range(len(answer_key)):
    ans = answers[i].strip().upper()
    if ans == answer_key[i]: correct += 1
    elif ans != "": wrong += 1
score = (correct * 2) - (wrong * 0.5)
print(f"Final Score: {score}")
```

CASE STUDY 6: TEXT COMPRESSION USING CHARACTER RUNS

1. PROBLEM STATEMENT

Implement Run-Length Encoding (RLE) text compression in Python to replace consecutive identical characters with character-count pairs.

2. FLOWCHART (STANDARD SYMBOLS - MONOCHROME)



3. PYTHON SOURCE CODE

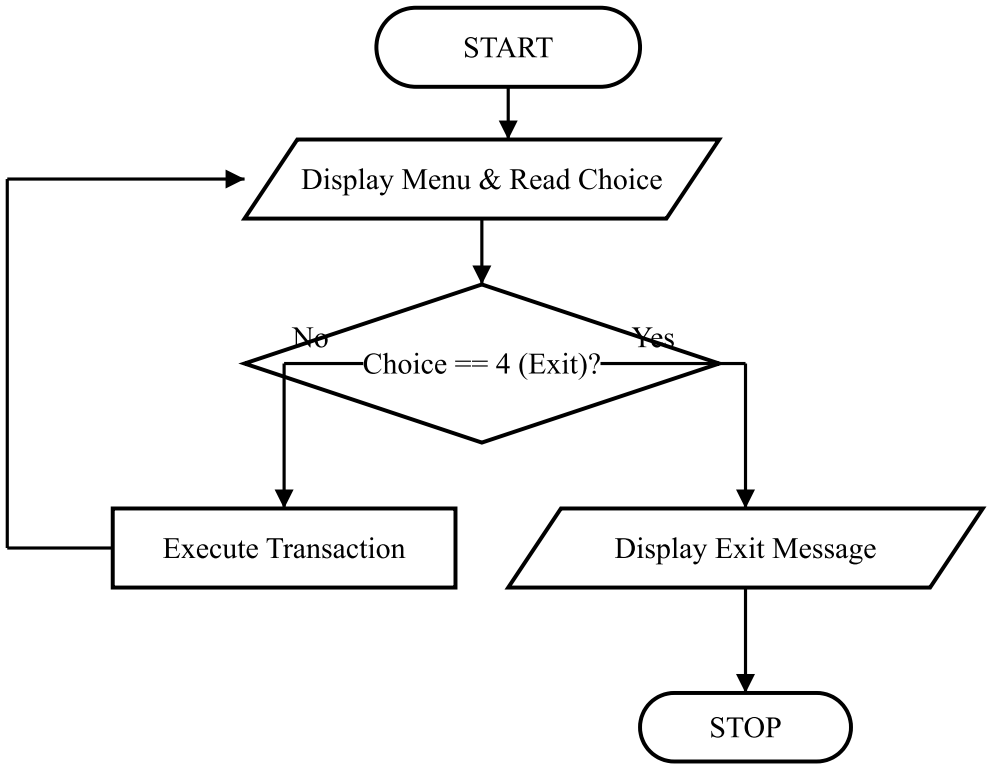
```
text = input("Enter string to compress: ")
compressed = ""
i = 0
while i < len(text):
    count = 1
    while i + 1 < len(text) and text[i] == text[i + 1]:
        count += 1; i += 1
    compressed += text[i] + str(count)
    i += 1
print("Compressed (RLE):", compressed)
```

CASE STUDY 7: ATM MENU-DRIVEN BANKING SIMULATION

1. PROBLEM STATEMENT

Create an interactive menu-driven ATM simulation supporting balance inquiry, deposit, withdrawal, and exit operations.

2. FLOWCHART (STANDARD SYMBOLS - MONOCHROME)



3. PYTHON SOURCE CODE

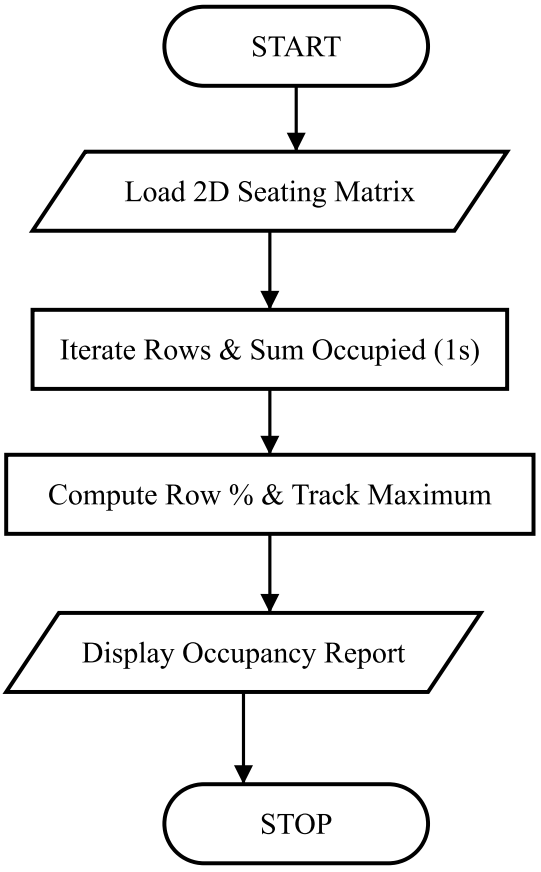
```
balance = 5000.0
while True:
    print("\n1. Check Balance 2. Deposit 3. Withdraw 4. Exit")
    choice = int(input("Select option: "))
    if choice == 1: print("Balance: Rs.", balance)
    elif choice == 2: balance += float(input("Deposit amt: "))
    elif choice == 3:
        amt = float(input("Withdraw amt: "))
        if amt <= balance: balance -= amt
        else: print("Insufficient funds!")
    elif choice == 4: print("Exiting. Thank you!"); break
```

CASE STUDY 8: NESTED LIST CLASSROOM OCCUPANCY ANALYSIS

1. PROBLEM STATEMENT

Analyze classroom seating occupancy data stored as 2D nested lists to calculate row-wise occupancy rates and identify crowded rows.

2. FLOWCHART (STANDARD SYMBOLS - MONOCHROME)



3. PYTHON SOURCE CODE

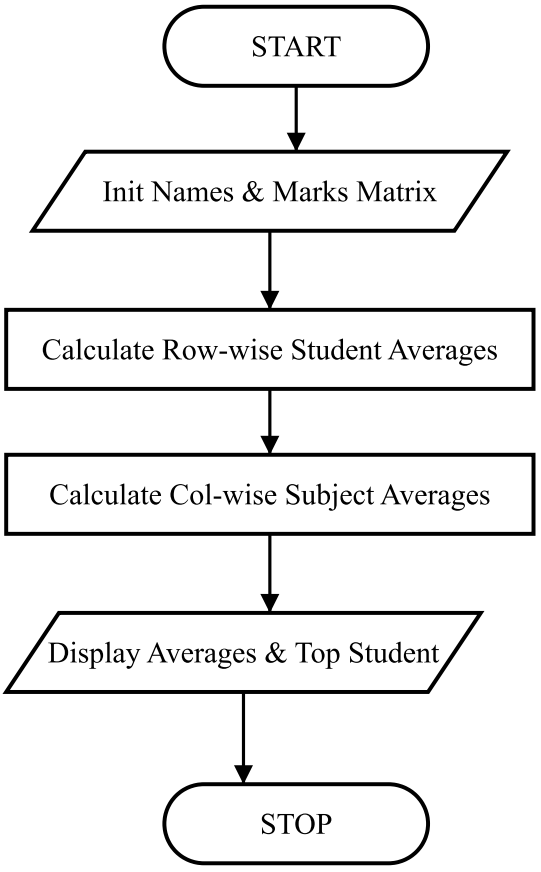
```
seating = [[1, 0, 1, 1], [1, 1, 1, 1], [0, 0, 1, 0]]
max_occupied = -1; crowded_row = 0
for i in range(len(seating)):
    occupied = sum(seating[i])
    pct = (occupied / len(seating[i])) * 100
    print(f"Row {i+1}: {pct:.2f}% occupied")
    if occupied > max_occupied:
        max_occupied = occupied; crowded_row = i + 1
print("Most Crowded Row:", crowded_row)
```

CASE STUDY 9: STUDENT MARKS MATRIX ANALYSIS

1. PROBLEM STATEMENT

Store student assessment marks in a matrix (nested list) and calculate student-wise averages, subject-wise averages, and identify top performers.

2. FLOWCHART (STANDARD SYMBOLS - MONOCHROME)



3. PYTHON SOURCE CODE

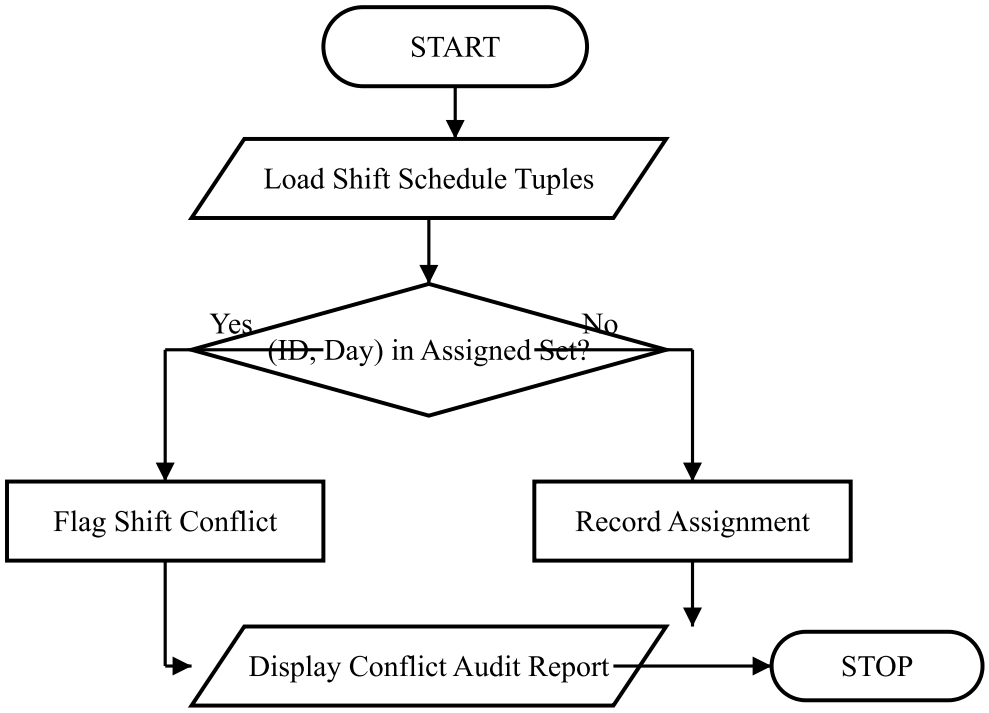
```
students = ["Asha", "Ravi", "Noor"]; subjects = ["Python", "Maths"]
marks = [[88, 76], [67, 82], [94, 89]]
averages = []
for i in range(len(students)):
    avg = sum(marks[i]) / len(marks[i])
    averages.append(avg)
    print(f"{students[i]} Avg: {avg:.2f}")
top_idx = averages.index(max(averages))
print("Top Performer:", students[top_idx])
```

CASE STUDY 10: TUPLE-BASED EMPLOYEE SHIFT CONFLICT DETECTION

1. PROBLEM STATEMENT

Process employee shift schedules stored as immutable tuples to detect scheduling conflicts where an employee is assigned overlapping shifts on the same day.

2. FLOWCHART (STANDARD SYMBOLS - MONOCHROME)



3. PYTHON SOURCE CODE

```
schedules = [
    ("E101", "Monday", "Morning"),
    ("E101", "Monday", "Night"),
    ("E102", "Tuesday", "Evening")
]
assigned = set(); conflicts = []
for emp_id, day, shift in schedules:
    key = (emp_id, day)
    if key in assigned: conflicts.append((emp_id, day, shift))
    else: assigned.add(key)
print("Shift Conflicts Detected:", conflicts)
```

PROJECT CONCLUSION

This report successfully implemented and verified ten Python case studies covering foundational programming concepts. By integrating standard ANSI flowchart visual elements without external colors, all technical workflows remain clean, consistent, and fully printable in standard black-and-white formats.